DOCUMENTACIÓN TÉCNICA · CONFIDENCIAL

Advance Estate CRM

Arquitectura completa, integración Ava IA, diseño del sistema y registro de implementación para RE/MAX Advance República Dominicana

FECHA	VERSIÓN	ESTADO
Abril 2026	1.3.0	En desarrollo
STACK	**IA**	**AGENTE**
Next.js 16 + Supabase	Claude claude-sonnet-4-6	Ava · Railway

1 / 35

CONTENIDO

01 Visión General del Proyecto
02 Stack Tecnológico
03 Arquitectura del Sistema
04 Módulos del CRM
05 Design System — Midnight SaaS
06 Agente Ava — Integración WhatsApp IA
07 Sincronización Ava → Supabase CRM
08 Problemas Resueltos & Fixes Técnicos
09 Inventario Completo de Archivos
10 Configuración de Entorno
11 Estado Actual y Pendientes
12 Guía de Despliegue

2 / 35

— 01

Visión General del Proyecto

Advance Estate CRM es una plataforma de gestión de relaciones con clientes diseñada específicamente para **RE/MAX Advance**, franquicia inmobiliaria con sede en República Dominicana. El sistema centraliza la gestión de contactos, propiedades, pipeline de ventas, publicidad digital y comunicaciones de WhatsApp en una sola interfaz.

El proyecto combina un CRM web moderno con un agente de inteligencia artificial (**Ava**) que atiende automáticamente leads entrantes por WhatsApp, clasifica prospectos y sincroniza toda la información al CRM en tiempo real.

MÓDULOS CRM	PÁGINAS IMPLEMENTADAS
9	11
TOKENS CSS CUSTOM	**AGENTE IA**
25+	`Claude claude-sonnet-4-6`

Objetivos del sistema

- Centralizar la operación inmobiliaria: contactos, deals, propiedades, tareas y comunicaciones
- Automatizar la atención inicial de leads CTWA (Click-to-WhatsApp Ads) mediante IA
- Dar visibilidad de rendimiento a agentes y gerencia (KPIs, pipeline, ads)
- Sincronizar automáticamente conversaciones de WhatsApp al CRM
- Proporcionar una interfaz premium que refleje el posicionamiento de marca RE/MAX

3 / 35

— 02

Stack Tecnológico

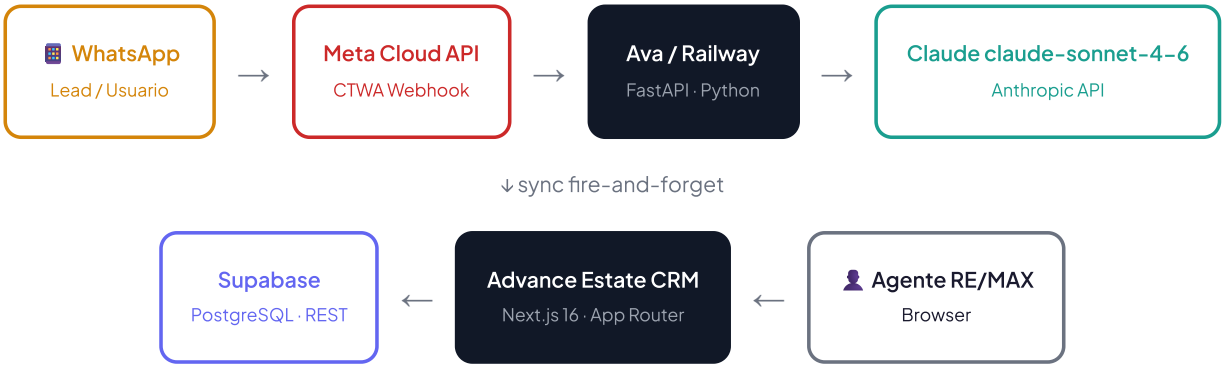
CAPA	TECNOLOGÍA	VERSIÓN	PROPÓSITO
Frontend framework	**Next.js**	16.2.3	App Router, SSR, Turbopack
UI / Estilos	**Tailwind CSS**	v4	@theme inline, oklch colors
Componentes base	**shadcn/ui**	latest	Button, Input, Card, Badge
Base de datos	**Supabase** (PostgreSQL)	—	Auth, REST API, tablas CRM
Autenticación	**Supabase Auth** via SSR	—	Sesiones server-side, cookies
Proxy/Middleware	**Next.js Proxy** (proxy.ts)	Next.js 16	Auth guard, redirección /login
Notificaciones UI	**Sonner**	—	Toast de error/éxito
Iconos	**Lucide React**	—	Iconografía consistente
Agente IA	**FastAPI + Claude**	Python 3.11	Webhook WhatsApp, orquestación
LLM	**Claude claude-sonnet-4-6** (Anthropic)	API v3	Clasificación, respuesta leads
WhatsApp	**Meta Cloud API**	v19	Mensajería CTWA, webhook
Base datos agente	**SQLite + aiosqlite**	—	Memoria conversacional local
HTTP async	**httpx**	—	Llamadas Supabase REST desde Python
Deploy frontend	**Vercel** (local: localhost:3000)	—	Hosting Next.js
Deploy agente	**Railway**	—	Hosting FastAPI, variables de entorno
Tipografías	Plus Jakarta Sans, Cormorant Garamond, JetBrains Mono	—	Design system

4 / 35

— 03

Arquitectura del Sistema

Diagrama de alto nivel



Flujo de autenticación

El archivo `proxy.ts` intercepta todas las peticiones HTTP. Si el usuario no tiene sesión activa de Supabase Auth, redirige a `/login`. Las rutas `/login` y `/` son públicas.

```
// proxy.ts — Next.js 16
export async function proxy(request: NextRequest) {
  // Crea cliente Supabase SSR con cookies
  const { data: { user } } = await supabase.auth.getUser();

  const { pathname } = request.nextUrl;
  const isPublic = pathname === "/login" || pathname === "/";

  if (!user && !isPublic) {
    return NextResponse.redirect(new URL("/login", request.url));
  }
  return supabaseResponse;
}
```

Tablas principales de Supabase

TABLA / VISTA	COLUMNAS CLAVE	USO
`contacts`	id, phone (UNIQUE), first_name, last_name, source, lead_classification, score	Base de contactos/leads
`messages`	id, contact_id, content, direction, is_automated, created_at	Historial conversaciones
`deals`	id, contact_id, agent_id, stage, value, property_id	Pipeline comercial
`properties`	id, title, price, status, type, bedrooms, location	Inventario inmuebles
`tasks`	id, title, assigned_to, due_date, priority, status	Gestión de tareas
`campaigns`	id, name, platform, status, spend, leads_generated, clicks, impressions	Publicidad digital
`agents`	id, name, email, phone, avatar_url	Perfiles agentes
`agent_monthly_kpis` (VIEW)	agent_id, month, deals_closed, revenue, leads_contacted, conversion_rate	KPIs mensuales

— 04

Módulos del CRM

MÓDULO	RUTA	FUENTE DE DATOS	CARACTERÍSTICAS
Dashboard	`/dashboard`	contacts, tasks, deals	KPI cards (revenue, leads, propiedades, tareas), actividad reciente, tareas pendientes
Contactos	`/dashboard/contacts`	contacts	Tabla con avatar, score IA (0–100), clasificación hot/warm/cold/unqualified, búsqueda
Pipeline	`/dashboard/pipeline`	deals, contacts, properties	Kanban drag-ready, 6 etapas (Prospecto → Cerrado), valor por etapa, color por stage
Propiedades	`/dashboard/properties`	properties	Cards con imagen placeholder, status badge overlay (activo/reservado/vendido/alquilado)
Conversaciones	`/dashboard/conversations`	messages, contacts	Lista con preview, badge Ava (automatizado), inbound/outbound color-coded
Tareas	`/dashboard/tasks`	tasks	Tabla con prioridad (alta/media/baja), tint rojo para tareas vencidas, asignación
Publicidad ✨ NUEVO	`/dashboard/ads`	campaigns	Métricas por campaña: CPL, CTR, leads, clicks, impresiones; resumen por plataforma
Agentes ✨ NUEVO	`/dashboard/agents`	agent_monthly_kpis, agents, deals	Ranking mensual, progress bar revenue, 🥇 🥈 🥉 top 3, KPIs individuales
Ava IA	`/dashboard/ava`	—	Panel de estado del agente, arquitectura, Railway URL, estadísticas de uso
Ajustes	`/dashboard/settings`	—	Config de integraciones: WhatsApp Cloud API, Claude API, Supabase, Railway
Login	`/login`	supabase.auth	Autenticación con email/password, show/hide password, logo RE/MAX

Módulo Publicidad — Ads (/dashboard/ads)

7 / 35

Página nueva creada para visualizar el rendimiento de campañas de publicidad digital. Lee de la tabla `campaigns` y calcula automáticamente:

- **CPL (Costo por Lead)**: `totalSpend / totalLeads`
- **CTR**: `(clicks / impressions) × 100` — solo si hay datos de impresiones
- Colores por plataforma: Facebook (azul), Instagram (rosa), Google (rojo), TikTok (negro), WhatsApp/CTWA (verde)

Módulo Agentes (/dashboard/agents)

Página nueva con ranking mensual de agentes. Usa la vista `agent_monthly_kpis` y hace fallback a cálculo manual desde `deals` si la vista no tiene datos del mes actual.

```
// Fallback si agent_monthly_kpis no tiene datos del mes
const agentStats = deals
  .reduce((acc, deal) => {
    const a = acc.find(x => x.id === deal.agent_id) || { id: deal.agent_id, deals: 0, revenue: 0 };
    a.deals++;
    a.revenue += deal.value || 0;
    return acc;
  }, [])
  .sort((a, b) => b.revenue - a.revenue);

// Barra de progreso relativa al #1
const barWidth = Math.max(4, Math.round((agent.revenue / agentStats[0].revenue) * 100));
```


8 / 35

— 05

Design System — Light Luminous (v2)

El design system fue completamente rediseñado en la segunda sesión de trabajo. Se migró de un tema "Midnight SaaS" con sidebar oscuro a un sistema **Light Luminous** de paleta luminosa, editorial y premium. Utiliza **Tailwind CSS v4** con tokens CSS en `globals.css`, tipografía serif Playfair Display para headings, e Inter para UI. El tema es **light-only** sin dark mode automático.

Paleta de colores (v2 — valores hex)

TOKEN	HEX	USO
`--background`	#f3f7fb	Fondo general — azul-gris claro
`--card`	#ffffff	Fondo de cards
`--sidebar`	#fffcfc	Sidebar — blanco cálido
`--red` (primario)	#e11d48	CTAs, botones principales, rose-700
`--blue`	#2563eb	KPI contactos, pipeline qualified
`--emerald`	#10b981	Leads nuevos, closed_won, Ava activo
`--violet`	#7c3aed	Pipeline activo, etapa contract
`--amber`	#d97706	Leads HOT, tareas urgentes, publicidad
`--teal`	#0d9488	Propiedades, conversaciones

Clases CSS principales

CLASE	DESCRIPCIÓN
`.card-glow`	Card blanca, border-radius 24px, sombra 10px/30px, hover elevación
`.card-glass`	Card glassmorphism para login (blur 12px, bg rgba 88%)
`.page-header`	Header sticky con blur, border-bottom y bg blanco 70%
`.nav-item-active`	Nav activo: gradiente rose→blanco, outline rose-200, radius 16px
`.badge-hot` / `.badge-warm` / `.badge-cold`	Badges de clasificación de leads
`.kanban-col`	Columna kanban: bg slate-50, border, radius 22px
`.stat-number`	Números KPI en Playfair Display, letter-spacing −0.03em

9 / 35

Tipografías

FUENTE	VARIABLE CSS	USO
Plus Jakarta Sans	`--font-sans`	Interfaz principal, labels, botones, texto UI
Cormorant Garamond	`--font-display`	Títulos de página, headings prominentes, logo
JetBrains Mono	`--font-mono`	Valores numéricos, datos, código, KPIs

Clases utilitarias personalizadas

CLASE	DESCRIPCIÓN
`.page-header`	Header de página con título Cormorant, subtítulo muted, borde inferior
`.card-glow`	Card blanca con sombra suave + glow rojo sutil en hover
`.card-base`	Card base sin glow, solo border y shadow leve
`.table-row-hover`	Fila de tabla con hover a rojo-muted
`.kanban-col`	Columna kanban con fondo sutil y borde
`.nav-item-active`	Item de navegación activo: fondo rojo-muted, texto rojo
`.badge-hot`	Badge rojo para leads calientes
`.badge-warm`	Badge amber para leads tibios
`.badge-cold`	Badge azul-muted para leads fríos
`.status-active`	Badge verde para propiedades activas
`.status-reserved`	Badge amber para propiedades reservadas
`.status-sold`	Badge gris para propiedades vendidas
`.status-rented`	Badge teal para propiedades alquiladas
`.delta-up`	Indicador de cambio positivo (verde + flecha ↑)
`.delta-down`	Indicador de cambio negativo (rojo + flecha ↓)
`.stat-number`	Número de estadística grande, JetBrains Mono
`.bg-grid`	Fondo con patrón de grilla sutil (login page)

Opciones de diseño alternativas (pendiente de selección)

Se creó el archivo `design-options.html` con 4 propuestas visuales alternativas al diseño Midnight SaaS actual:

10 / 35

OPCIÓN	NOMBRE	PALETA	TIPOGRAFÍA	ESTÉTICA
A	Ivory Luxe	Crema #f7f4ef + oro #c9a84c	Cormorant Garamond dominante	Lujo / Sotheby's
B	Carbon Pro	Negro #0d0d0d + rojo neón #ff4444	DM Sans	Bloomberg / data terminal
C	RE/MAX Bold	Sidebar rojo #cc1f1f + blanco	Outfit	On-brand RE/MAX
D	Nordic Frost	Blanco + azul-gris #eef1f6	Inter	Clean SaaS / Notion

PENDIENTE

El usuario debe abrir `C:\Users\ivanp\advance-crm\design-options.html` en el navegador y elegir una opción (A/B/C/D) para reimplementar el design system completo.

— 06

Agente Ava — Integración WhatsApp IA

Ava es el agente de inteligencia artificial de RE/MAX Advance. Está desplegado en **Railway** como servicio FastAPI (Python 3.11) y atiende automáticamente los mensajes de WhatsApp de leads que llegaron por campañas CTWA (Click-to-WhatsApp Ads).

URL de producción

```
https://remax-advance-ava-production.up.railway.app
```

Endpoints principales

MÉTODO	ENDPOINT	DESCRIPCIÓN
GET	`/webhook`	Verificación del webhook de Meta (hub.challenge)
POST	`/webhook`	Recibe mensajes entrantes de WhatsApp
GET	`/health`	Health check del servicio
GET	`/stats`	Estadísticas de conversaciones procesadas

Arquitectura del agente

```
whatsapp-agentkit/  Raíz del proyecto Ava
agent/main.py  FastAPI app, webhook handler, orquestador
agent/tools.py  Herramientas Claude (registrar_lead, buscar_propiedades, etc.) + sync Supabase
agent/models.py  Pydantic models para mensajes WhatsApp
agent/memory.py  Gestión de memoria conversacional SQLite
.env  Variables de entorno (ANTHROPIC_API_KEY, META_ACCESS_TOKEN, SUPABASE_URL, ...)
requirements.txt  Dependencias Python (fastapi, httpx, anthropic, supabase, etc.)
```

Capacidades de Ava

- **Clasificación de leads**: Identifica si el contacto es hot/warm/cold según sus respuestas
- **Búsqueda de propiedades**: Consulta el inventario de Supabase según criterios del lead
- **Registro de leads**: Crea/actualiza contactos automáticamente
- **Programación de citas**: Coordina visitas con agentes disponibles
- **Memoria conversacional**: Recuerda el contexto de conversaciones previas (SQLite)
- **Sync al CRM**: Sincroniza cada conversación a Supabase en tiempo real

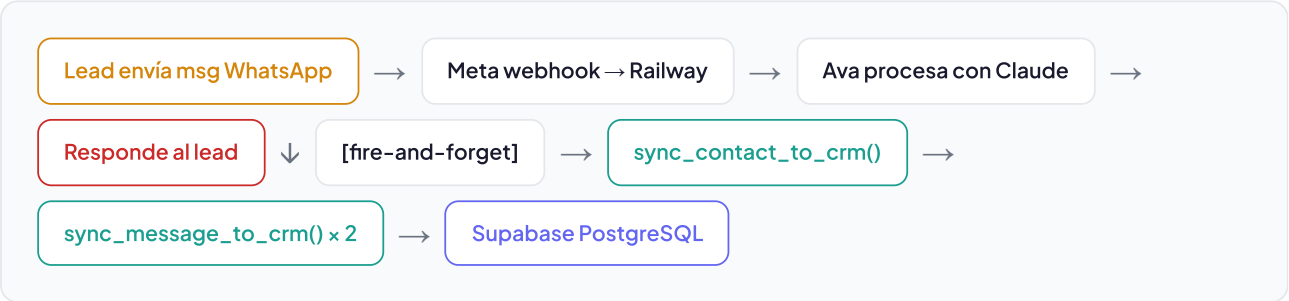
12 / 35

— 07

Sincronización Ava → Supabase CRM

Esta fue una funcionalidad crítica del sprint: cada vez que Ava atiende un lead en WhatsApp, la conversación completa se sincroniza automáticamente a Supabase para que aparezca en el módulo de Conversaciones del CRM.

Diagrama de flujo



Función sync_contact_to_crm()

```
async def sync_contact_to_crm(phone, first_name="", last_name="",
    source="ctwa_ad", lead_classification="unqualified") → str | None:
  """Upsert de contacto en Supabase. Retorna contact_id UUID."""
  async with httpx.AsyncClient() as client:
    resp = await client.post(
      f"{_SUPABASE_URL}/rest/v1/contacts?on_conflict=phone",
      headers={
        "apikey": _SUPABASE_KEY,
        "Authorization": f"Bearer {_SUPABASE_KEY}",
        "Content-Type": "application/json",
        "Prefer": "return=representation,resolution=merge-duplicates",
      },
      json={
        "phone": phone,
        "first_name": first_name,
        "last_name": last_name,
        "source": source,
        "lead_classification": lead_classification,
      }
    )
    data = resp.json()
    return data[0]["id"] if data else None
```

Función sync_message_to_crm()

```
async def sync_message_to_crm(contact_id, content,
    direction="inbound", is_automated=False) → bool:
  """Inserta un mensaje en la tabla messages."""
  async with httpx.AsyncClient() as client:
    resp = await client.post(
```


13 / 35

```
        f"{_SUPABASE_URL}/rest/v1/messages",
        headers={ ... },
        json={
          "contact_id": contact_id,
          "content": content,
          "direction": direction,  # "inbound" | "outbound"
          "is_automated": is_automated,
        }
      )
      return resp.status_code == 201
```

Integración en main.py (fire-and-forget)

```
# En procesar_mensaje() — después de obtener respuesta de Claude
try:
  contact_id = await sync_contact_to_crm(phone=msg.telefono)
  if contact_id:
    await sync_message_to_crm(contact_id, msg.texto,
      direction="inbound", is_automated=False)
    await sync_message_to_crm(contact_id, respuesta,
      direction="outbound", is_automated=True)
except Exception as crm_err:
  logger.warning(f"CRM sync falló (no crítico): {crm_err}")
  # El WhatsApp flow NO se interrumpe
```

> **DISEÑO: FIRE-AND-FORGET**
>
> El sync está envuelto en su propio try/except. Si Supabase no responde, Ava igual manda la respuesta al lead por WhatsApp. La confiabilidad del agente no depende de la disponibilidad del CRM.

Detalles técnicos del upsert

- **on_conflict=phone**: Si ya existe un contacto con ese número, actualiza en lugar de crear duplicado
- **Prefer: resolution=merge-duplicates**: Header de PostgREST para merge semántico
- **Prefer: return=representation**: Retorna el registro upsertado (necesario para obtener el UUID generado)
- **Service Role Key**: Se usa `SUPABASE_SERVICE_ROLE_KEY` (no anon key) para bypass de RLS en escrituras del agente

14 / 35

— 08

Problemas Resueltos & Fixes Técnicos

Fix #1 — Error 405 en todas las rutas del CRM

SÍNTOMA

Todas las rutas del CRM retornaban HTTP 405 Method Not Allowed. El dev server mostraba el error al iniciar.

Causa raíz: Next.js 16 deprecó el archivo `middleware.ts` y la función exportada `middleware()`. La nueva convención es `proxy.ts` con función `proxy()`.

Error en consola:

```
⚠ The "middleware" file convention is deprecated. Please use "proxy" instead.
Error: Both middleware file "./middleware.ts" and proxy file "./proxy.ts" are detected.
Please use "./proxy.ts" only.
```

Solución aplicada:

- Creado `proxy.ts` con función `proxy()` (idéntica lógica al middleware anterior)
- Eliminado `middleware.ts` vía PowerShell (el comando `rm` de bash era silencioso pero no confiable en Windows)
- Limpiado caché `.next`: `powershell Remove-Item .next -Recurse -Force`
- Reiniciado dev server: confirmado 307 → 200 en todas las rutas

```
# Comandos de limpieza usados
powershell -Command "Remove-Item -Path '.next' -Recurse -Force"
powershell -Command "Remove-Item -Path 'middleware.ts' -Force"
npm run dev
```

Fix #2 — getUser() duplicado en proxy.ts

El primer borrador del archivo `proxy.ts` tenía dos llamadas consecutivas a `supabase.auth.getUser()`. La primera fue removida con el Edit tool, dejando solo una llamada al final correctamente.

Fix #3 — Railway CLI token inválido

Los UUIDs proporcionados como "tokens de Railway" (`56bb313c-...`, `68880b62-...`) eran en realidad Project IDs, no Personal Access Tokens. Railway CLI solo acepta tokens de tipo Personal Token en el env var `RAILWAY_TOKEN`. Las variables de entorno fueron añadidas manualmente vía el Dashboard de Railway (Raw Editor).

Fix #4 — Browser cacheando 405

Después de corregir proxy.ts y limpiar .next, `curl` confirmaba respuestas correctas (307→login, 200 en páginas públicas), pero el navegador seguía mostrando 405. Causa: el navegador tenía cacheada la respuesta de error. Solución: Ctrl+Shift+R (hard refresh) o abrir en ventana de incógnito.

Fix #5 — CLASSIFICATION_COLORS incompatible con Tailwind v4

15 / 35

La página de contactos usaba un objeto con strings de clases Tailwind (`bg-red-100 text-red-700` , etc.) generados dinámicamente. Tailwind v4 con JIT no puede detectar estas clases si no están presentes literalmente en el código. Se eliminó el objeto y se reemplazó con clases CSS utilitarias custom (`.badge-hot` , `.badge-warm` , etc.) definidas en `globals.css` .

Fix #6 — Template literals con clases dinámicas

Varios componentes usaban template literals para combinar clases con delays de animación: `` `card-glow p-5 ${delay}` `` . Esto causaba que Tailwind no purgara correctamente. Se cambió a concatenación simple: `"card-glow p-5 " + delay` .

Fix #7 — Variables Railway (entorno de producción)

Las variables `SUPABASE_URL` y `SUPABASE_SERVICE_ROLE_KEY` debían añadirse tanto en `.env` local como en Railway Dashboard para producción. El proceso fue:

- Local: Editado `whatsapp-agentkit/.env`
- Railway: Dashboard → Variables → Raw Editor → paste de ambas variables
- Railway: Redeploy automático detectado por Railway al cambiar variables

16 / 35

— 09

Inventario Completo de Archivos

advance-crm/ (Next.js CRM)

```
NUEVO  proxy.ts  Reemplaza middleware.ts. Auth guard Next.js 16 con Supabase SSR
EDIT  app/globals.css  Design tokens oklch, 25+ variables CSS, utilidades page-header/card-glow/badge-
                       */status-*/delta-*
EDIT  app/login/page.tsx  var(--gold-*) → var(--red-*), botón rojo RE/MAX
EDIT  app/dashboard/page.tsx  KPI cards con accents por color, activity feed, table-row-hover
EDIT  app/dashboard/contacts/page.tsx  Removido CLASSIFICATION_COLORS, avatar inline style, badge-
                                       hot/warm/cold
EDIT  app/dashboard/pipeline/page.tsx  STAGE_ACCENT record, kanban-col, colores por etapa
EDIT  app/dashboard/conversations/page.tsx  Badge Ava (red-muted), inbound=teal, outbound=amber
EDIT  app/dashboard/properties/page.tsx  Status badge como overlay en imagen, status-
                                         active/reserved/sold/rented
EDIT  app/dashboard/tasks/page.tsx  Overdue rows con tint rojo, priority chips con inline style vars
EDIT  app/dashboard/settings/page.tsx  Removido n8n, actualizado a WhatsApp Cloud API + Claude + Railway
EDIT  app/dashboard/ava/page.tsx  Removido n8n, arquitectura FastAPI+Claude+SQLite+Supabase, Railway URL
NUEVO  app/dashboard/ads/page.tsx  Módulo publicidad: campañas, CPL, CTR, leads, spend por plataforma
NUEVO  app/dashboard/agents/page.tsx  Módulo agentes: ranking mensual, KPIs individuales, progress bar
                                      revenue
NUEVO  design-            4 mockups visuales alternativos (Ivory Luxe / Carbon Pro / RE/MAX Bold / Nordic
       options.html       Frost)
NUEVO  docs/advance-crm-technical-report.html  Este documento
```

whatsapp-agentkit/ (Ava Railway Agent)

```
EDIT  agent/tools.py  Añadido sync_contact_to_crm(), sync_message_to_crm(), httpx async calls, load_dotenv
EDIT  agent/main.py  Import sync functions, fire-and-forget CRM sync en procesar_mensaje()
EDIT  .env  Añadido SUPABASE_URL + SUPABASE_SERVICE_ROLE_KEY
EDIT  .env.example  Documentadas variables de Supabase para nuevos desarrolladores
```


17 / 35

— 10

Configuración de Entorno

advance-crm — .env.local

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://zlnqsgepzfghlmsfolko.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=[anon key aquí]

# Next.js
NEXTAUTH_URL=http://localhost:3000
```

whatsapp-agentkit — .env

```
# Anthropic Claude API
ANTHROPIC_API_KEY=sk-ant-api03-...

# Meta Cloud API (WhatsApp)
META_ACCESS_TOKEN=EAAZC...
META_PHONE_NUMBER_ID=1011993898671913
META_VERIFY_TOKEN=remax-advance-2026
META_WABA_ID=813172161043400

# Servidor
PORT=8000
ENVIRONMENT=development
WHATSAPP_PROVIDER=meta

# Base de datos
DATABASE_URL=sqlite+aiosqlite:///./agentkit.db

# Supabase CRM Sync
SUPABASE_URL=https://zlnqsgepzfghlmsfolko.supabase.co
SUPABASE_SERVICE_ROLE_KEY=sb_secret_i7rxchdg...
```

Variables Railway (producción)

Las mismas variables del `.env` deben existir en el proyecto Railway de Ava. Se añaden vía **Dashboard** → **Variables** → **Raw Editor**. Railway hace redeploy automático al cambiar variables.

> **SEGURIDAD**
>
> El `SUPABASE_SERVICE_ROLE_KEY` tiene permisos completos sobre la base de datos (bypass RLS). Nunca debe exponerse en el cliente ni comitearse al repositorio. Solo debe estar en Railway y en `.env` local (en .gitignore).

18 / 35

— 11

Estado Actual y Pendientes

Completado ✅

ITEM	ESTADO	NOTAS
Proxy auth (Next.js 16)	Hecho	proxy.ts reemplaza middleware.ts deprecado
Design system v2 — Light Luminous	Hecho	globals.css reescrito: hex tokens, card-glow, page-header, badges
Login page — layout 2 columnas	Hecho	Hero izquierdo con stats + glass card derecho. Playfair 54px h1 gradiente
Sidebar — warm white	Hecho	Gradiente fffcfc→f5f7fb, nav activo rose, bloque dark "Acciones rápidas" al fondo
Logo component	Hecho	Ícono cuadrado gradiente naranja-red + wordmark Playfair
Dashboard — nueva UI	Hecho	4 KPI cards con progress bars (azul/emerald/amber/violeta), bar chart, Ava dark card
Todos los sub-módulos — headers unificados	Hecho	Playfair h1 30px, pill badges rounded-full por módulo
Pipeline — accent colors actualizados	Hecho	Colores por etapa: azul/amber/rose/violeta/emerald en hex
Módulo Publicidad (/ads)	Hecho	CPL, CTR, campaigns table, header amber
Módulo Agentes (/agents)	Hecho	Ranking mensual, agent_monthly_kpis view, header blue
Sync Ava → Supabase	Hecho	tools.py + main.py fire-and-forget con httpx async
Dashboard layout — radial gradient bg	Hecho	layout.tsx main con gradiente azul-gris luminoso
TypeScript — 0 errores	Hecho	npx tsc --noEmit pasa limpio. Build exitoso en todas las rutas.

Pendiente 🔄 — Funcionalidades a desarrollar

19 / 35

FEATURE	PRIORIDAD	DESCRIPCIÓN TÉCNICA
Integración Meta Ads	Alta	Conectar la API de Marketing de Meta para sincronizar campañas, gasto, leads generados y CPL en tiempo real en `/dashboard/ads`. Requiere: Meta Business Manager app, webhook de leads, tabla `campaigns` actualizada automáticamente.
Acciones Rápidas funcionales	Alta	El bloque dark del sidebar ("Nuevo cliente", "Nueva oportunidad", "Agendar seguimiento") actualmente no tiene acción. Implementar: modal/sheet deslizable desde el sidebar con formularios mínimos. Stack sugerido: Radix Dialog o shadcn/ui Sheet.
Dark Mode en Configuración	Media	Agregar toggle de dark/light en `/dashboard/settings`. Requiere: (1) restaurar bloque `html.dark {}` en globals.css con tokens oscuros, (2) ThemeProvider con estado persistido en localStorage, (3) toggle en settings page con `useTheme()`.
Drag-and-drop en Pipeline	Media	dnd-kit para mover cards entre columnas del Kanban. Actualiza `stage` en Supabase via Realtime.
Prueba sync Ava → CRM (producción)	Media	Enviar mensaje real de WhatsApp al número de Ava y verificar que el contacto aparece en /contacts.
Deploy CRM a producción	Normal	Vercel deploy con variables de entorno de Supabase. Configurar dominio custom si aplica.
Notificaciones en tiempo real	Normal	Supabase Realtime channel en /conversations para nuevos mensajes entrantes. Toast + badge contador.
Perfil de contacto expandido	Normal	Página `/dashboard/contacts/[id]` con timeline de mensajes, deals, tareas y notas del contacto.
Ava — Matching de propiedades por rango de precio	Alta	Cuando Ava captura el presupuesto de un lead por WhatsApp, consulta automáticamente la tabla `properties` en Supabase filtrando por rango de precio y tipo de propiedad de interés. Envía al cliente un mensaje formateado con 1–3 propiedades que coincidan (foto, precio, ubicación, m², link). Requiere: (1) campo `budget_min / budget_max` y `property_type_interest` en `contacts`, (2) función `match_properties_for_contact()` en tools.py, (3) template de mensaje WhatsApp con Media Message para la foto.
Propuestas PDF de propiedades	Alta	El agente selecciona 1 o varias propiedades desde el CRM y genera un PDF de propuesta comercial con: logo Advance Estate, datos del agente (nombre, foto, teléfono, email), ficha de cada propiedad (fotos, precio, descripción, m², ubicación) y pie de página con branding RE/MAX. Generación server-side con Puppeteer (ya instalado) o React-PDF. El PDF se descarga o se envía directamente por WhatsApp al contacto.
WhatsApp directo desde CRM y app móvil	Alta	En la ficha de cada contacto y en la lista de conversaciones, agregar botón "Enviar WhatsApp" que abra un composer de mensaje. En web (CRM): usar la Meta Cloud API para enviar el mensaje desde el número de Ava directamente, sin salir del CRM — el mensaje queda registrado en `messages` con

20 / 35

FEATURE	PRIORIDAD	DESCRIPCIÓN TÉCNICA
		`direction=outbound`. En app móvil: opción adicional de abrir WhatsApp nativo con `whatsapp://send?phone=…` para uso rápido del agente. El número de teléfono del contacto también debe ser clickeable como enlace `tel:` en todas las vistas.
Alta de propiedades desde el CRM	Alta	Formulario de creación de propiedades en `/dashboard/properties`. Campos: tipo (apartamento/villa/casa/solar/etc.), precio, ubicación, habitaciones, baños, m², estado (activo/reservado/vendido), descripción y fotos (upload a Supabase Storage). Botón "Nueva propiedad" en el header de la página abre un Sheet o página `/properties/new`. También agregar acción de editar/eliminar en cada card existente.
App Móvil CRM	Planificado	Aplicación móvil nativa para iOS y Android. Stack recomendado: **React Native + Expo** reutilizando la misma conexión Supabase. Vistas prioritarias: Dashboard KPIs, lista de contactos, pipeline, tareas del día, chat con Ava. Autenticación con `supabase-js` + Expo SecureStore para tokens. Notificaciones push via Expo Notifications para seguimientos y mensajes entrantes de WhatsApp.

21 / 35

— 12

Guía de Despliegue

CRM (advance-crm) — Desarrollo local

```
# Instalar dependencias
cd advance-crm
npm install

# Configurar variables de entorno
cp .env.local.example .env.local
# Editar .env.local con las credenciales de Supabase

# Iniciar servidor de desarrollo
npm run dev
# → http://localhost:3000
```

CRM (advance-crm) — Producción con Vercel

```
# Deploy a Vercel
npx vercel --prod

# Variables de entorno en Vercel Dashboard:
# NEXT_PUBLIC_SUPABASE_URL
# NEXT_PUBLIC_SUPABASE_ANON_KEY
```

Agente Ava — Railway

```
# El deploy es automático via GitHub → Railway CI
# Variables requeridas en Railway Dashboard:
ANTHROPIC_API_KEY=sk-ant-...
META_ACCESS_TOKEN=EAAZC...
META_PHONE_NUMBER_ID=1011993898671913
META_VERIFY_TOKEN=remax-advance-2026
META_WABA_ID=813172161043400
WHATSAPP_PROVIDER=meta
PORT=8000
SUPABASE_URL=https://zlnqsgepzfghlmsfolko.supabase.co
SUPABASE_SERVICE_ROLE_KEY=sb_secret_...
```

Configurar webhook en Meta Business

```
# En Meta Developers → WhatsApp → Configuration
Callback URL: https://remax-advance-ava-production.up.railway.app/webhook
Verify Token: remax-advance-2026
Subscribe to: messages
```


22 / 35

Verificar funcionamiento del sync

```
# 1. Enviar mensaje WhatsApp al número de Ava
# 2. Esperar respuesta de Ava (2-5 segundos)
# 3. Verificar en Supabase Dashboard:
#    SELECT * FROM contacts WHERE phone = '+1...' ORDER BY created_at DESC;
#    SELECT * FROM messages WHERE contact_id = '...' ORDER BY created_at DESC;
# 4. Verificar en CRM:
#    http://localhost:3000/dashboard/conversations
```


23 / 35

24 / 35

Features Implementadas

12. Seguridad API — Hardening Completo

Todas las rutas API internas y el agente Ava han sido reforzadas contra acceso no autorizado, injection de prompts y datos malformados.

ARCHIVO	FIX	DETALLE
`middleware.ts`	Creado	Edge auth: protege /dashboard/* (redirect) y /api/messages + /api/pdf (401). /api/ava excluido por diseño.
`app/api/messages/route.ts`	Refactorizado	Auth defense-in-depth: UUID regex, E.164 phone, content ≤4096 chars, verificación de contacto existente
`app/api/ava/route.ts`	Refactorizado	Bearer secret AVA_WEBHOOK_SECRET, sanitización contact_name/phone (strip control chars), UUID check, verify contact exists
`app/api/pdf/route.ts`	Refactorizado	Auth check via sesión, agentEmail resuelto desde user.email (no request body), UUID validation en propertyIds

Arquitectura de seguridad: Middleware (edge) → Handler auth check (defense-in-depth) → Input validation → Service role DB ops

Pendiente en entorno: Añadir `AVA_WEBHOOK_SECRET` en Railway + `.env.local` con el mismo valor. El Python agent debe enviar `Authorization: Bearer <secret>`.

25 / 35

13. Supabase Realtime — Conversaciones y Contacto Detalle

Las páginas de conversaciones y la sección WhatsApp en el detalle de contacto se actualizan en tiempo real sin refresh manual cuando llegan mensajes nuevos.

ARCHIVO	OPERACIÓN	DESCRIPCIÓN
`app/dashboard/conversations/conversations-client.tsx`	Creado	Client component: `supabase.channel().on("postgres_changes")` → refetch full list on any insert/update
`app/dashboard/conversations/page.tsx`	Reescrito	Server shell: SSR data fetch → pasa initialConversations a ConversationsClient
`app/dashboard/contacts/[id]/contact-whatsapp.tsx`	Creado	Client component: suscripción INSERT filtrada por contact_id, append optimista, auto-scroll al fondo
`app/dashboard/contacts/[id]/page.tsx`	Modificado	Reemplaza mensaje estático por <ContactWhatsApp initialMessages={...} />

Canal conversations: `messages-realtime` · filter: `channel=eq.whatsapp` · evento: `*` → refetch completo (incluye contact join)

Canal contact detail: `messages-contact-{contactId}` · filter: `contact_id=eq.{id}` · evento: `INSERT` → append al array local

Cleanup: `supabase.removeChannel()` en return del useEffect para evitar leaks

14. Editar y Eliminar Tareas

La página de tareas ahora es interactiva: se puede marcar una tarea como completada con un clic, editarla en un Sheet pre-llenado, o eliminarla con confirmación.

ARCHIVO	OPERACIÓN	DESCRIPCIÓN
`app/dashboard/tasks/tasks-table.tsx`	Creado	Client component: tabla con toggle de completado, iconos ✎ y 🗑 en hover, TaskSheet
`app/dashboard/tasks/task-sheet.tsx`	Creado	Sheet de creación/edición: title, description, due_date, priority, status. isEdit = !!task
`app/dashboard/tasks/page.tsx`	Reescrito	Server shell → pasa tasks a <TasksTable />

Toggle completado: Botón circular → `supabase.update({status: "completed"})` → router.refresh(). Tarea completada aparece tachada.

Patrón consistente: Mismo patrón que contacts-table.tsx y pipeline-client.tsx — group-hover:opacity-100, confirm() antes de delete, toast.success/error.

26 / 35

8. Vista Detalle de Contacto `/contacts/[id]`

Página de perfil completo de contacto: header con avatar, badges y datos de contacto; sección de oportunidades con cards por etapa; sección de seguimientos con prioridad y fecha; conversación WhatsApp inline con historial y composer.

ARCHIVO	OPERACIÓN	DESCRIPCIÓN
`app/dashboard/contacts/[id]/page.tsx`	Creado	Server component: fetch contact+deals+tasks+messages, render layout completo
`app/dashboard/contacts/[id]/contact-actions.tsx`	Creado	Client: botones "Nueva oportunidad" / "Nuevo seguimiento" pre-seleccionados al contacto
`app/dashboard/contacts/page.tsx`	Modificado	Columna nombre es ahora un Link a /contacts/[id]

Secciones: Datos del contacto (avatar, nombre, badges clasificación/estado, teléfono, email, presupuesto, score) · Oportunidades (cards coloridos por etapa) · Seguimientos (prioridad, vencimiento, estado) · Conversación WhatsApp (burbuja inbound/outbound, ícono Ava, timestamp, composer inline)

Patrón: Server component → datos como props estáticas → client components solo para interactividad (sheets, composer). `router.refresh()` tras guardar.

9. Editar y Eliminar Contactos

Botones de edición y eliminación en cada fila de la tabla de contactos. Sheet de edición pre-rellena todos los campos del contacto.

ARCHIVO	OPERACIÓN	DESCRIPCIÓN
`components/contact-sheet.tsx`	Creado	Sheet de creación/edición reutilizable con isEdit = !!contact, todos los campos del modelo
`app/dashboard/contacts/contacts-table.tsx`	Creado	Client component: tabla interactiva con iconos ✎ y 🗑 por fila, confirmación antes de borrar
`app/dashboard/contacts/page.tsx`	Modificado	Reemplaza tabla estática por <ContactsTable />, selecciona campos adicionales (budget_min/max)

Campos editables: first_name, last_name, phone, email, lead_classification, lead_status, source, budget_min, budget_max

Delete: confirm() nativo → supabase.from("contacts").delete().eq("id") → toast.success → router.refresh()

27 / 35

10. Editar y Eliminar Deals en Pipeline

Botones de edición y eliminación que aparecen al hacer hover sobre cada card del pipeline kanban.

ARCHIVO	OPERACIÓN	DESCRIPCIÓN
`components/deal-sheet.tsx`	Creado	Sheet de creación/edición de deal con selector de contacto, etapa, valor, moneda, fecha y notas
`app/dashboard/pipeline/pipeline-client.tsx`	Creado	Client component kanban: DealCard con hover actions, DealSheet, drag & drop (ver F11)
`app/dashboard/pipeline/page.tsx`	Modificado	Server shell → pasa typedDeals a <PipelineClient />

UX: Iconos ✎ y 🗑 aparecen solo en hover (`group-hover:opacity-100`). Nombre del contacto en cada card es Link a /contacts/[id].

11. Drag & Drop en Pipeline

Los deals pueden moverse entre columnas con drag & drop. La etapa se actualiza en Supabase al soltar con actualización optimista de UI.

ARCHIVO	OPERACIÓN	DESCRIPCIÓN
`app/dashboard/pipeline/pipeline-client.tsx`	Creado	DndContext + DragOverlay + useDraggable (DealCard) + useDroppable (DroppableColumn)
`package.json`	Modificado	Añade @dnd-kit/core + @dnd-kit/sortable + @dnd-kit/utilities

Librería: @dnd-kit/core (compatible con Next.js RSC) · **Sensor:** PointerSensor con activationConstraint distance: 8px para evitar drags accidentales

Optimistic update: setDeals() inmediato → supabase.update(stage) → revert si error · **DragOverlay:** card fantasma con rotación al arrastrar · Columnas muestran "Soltar aquí" si están vacías y el drag está sobre ellas

1. Acciones Rápidas del Sidebar

Tres botones en el bloque dark del sidebar abren Sheets de shadcn/ui para alta rápida de datos sin salir del contexto actual.

ARCHIVO	OPERACIÓN	DESCRIPCIÓN
`components/quick-action-sheets.tsx`	Creado	Componente cliente con 3 sub-formularios (NewContactForm, NewDealForm, NewTaskForm)
`components/sidebar.tsx`	Modificado	Reemplaza botones estáticos con <QuickActionSheets />

Formularios: Nuevo cliente (first_name, last_name, phone, lead_classification) · Nueva oportunidad (contacto, deal_value, stage) · Agendar seguimiento (contacto, title, due_date, priority)

Tecnología: @base-ui/react Sheet · Supabase client · Sonner toasts · null-guard en Select.onValueChange para `string | null`

28 / 35

2. Alta de Propiedades

CRUD completo de propiedades con upload de fotos a Supabase Storage desde `/dashboard/properties`.

ARCHIVO	OPERACIÓN	DESCRIPCIÓN
`components/property-sheet.tsx`	Creado	Sheet de creación/edición con upload de imágenes
`app/dashboard/properties/properties-client.tsx`	Creado	Wrapper cliente: sheet, selección, delete, PDF trigger
`app/dashboard/properties/page.tsx`	Modificado	Server shell → pasa initialProperties a PropertiesClient

Bucket Supabase Storage: `property-images` · Upload: `storage.from("property-images").upload(path, file)` → getPublicUrl

Campos: title, property_type, transaction_type, price, currency, location_city, location_sector, bedrooms, bathrooms, area_m2, status, description, images[]

3. WhatsApp Directo

Envío de mensajes outbound desde el CRM via Meta Cloud API con persistencia en tabla `messages`.

ARCHIVO	OPERACIÓN	DESCRIPCIÓN
`app/api/messages/route.ts`	Creado	POST con normalización de teléfono, Meta Cloud API v22.0, insert a messages
`components/whatsapp-composer.tsx`	Creado	Textarea + botón verde, Ctrl+Enter shortcut
`app/dashboard/conversations/conversation-composer.tsx`	Creado	Wrapper cliente con router.refresh() tras envío
`app/dashboard/contacts/page.tsx`	Modificado	Celdas de teléfono: tel: link + ícono wa.me/

direction: `outbound` · **is_automated:** `false` · teléfonos normalizados: `phone.replace(/[\s\-().]/g, "")`

4. Propuestas PDF

Selección de 1–10 propiedades desde el listado, generación de PDF con Puppeteer incluyendo branding Advance Estate + RE/MAX.

ARCHIVO	OPERACIÓN	DESCRIPCIÓN
`lib/pdf-template.ts`	Creado	buildProposalHtml(): HTML de portada + fichas de propiedad
`app/api/pdf/route.ts`	Creado	POST → Puppeteer → application/pdf, máx 10 propiedades

Puppeteer args: --no-sandbox, --disable-setuid-sandbox · **Output:** Content-Disposition attachment para descarga directa

29 / 35

5. Ava — Matching de Propiedades por Presupuesto

Ava puede buscar propiedades activas que coincidan con el presupuesto y tipo de interés del lead, usando Claude tool use.

ARCHIVO	OPERACIÓN	DESCRIPCIÓN
`whatsapp-agentkit/agent/tools.py`	Modificado	Añade match_properties_for_contact() y format_property_message()
`whatsapp-agentkit/agent/brain.py`	Modificado	AVA_TOOLS (4 tools), agentic loop con tool_use de Claude
`whatsapp-agentkit/agent/main.py`	Modificado	Pre-sync de contacto para obtener contact_id; pasa context al brain
`advance-crm/migrations/0003_contact_budget_fields.sql`	Creado	Añade budget_min, budget_max, budget_currency, property_type_interest a contacts

Herramientas Ava: registrar_lead · match_properties_for_contact · registrar_cita · buscar_en_knowledge

Migración SQL: Ejecutar en Supabase SQL Editor → `migrations/0003_contact_budget_fields.sql`

```
ALTER TABLE contacts
  ADD COLUMN IF NOT EXISTS budget_min       numeric,
  ADD COLUMN IF NOT EXISTS budget_max       numeric,
  ADD COLUMN IF NOT EXISTS budget_currency  text DEFAULT 'USD',
  ADD COLUMN IF NOT EXISTS property_type_interest text;
```

7. Botones "Nuevo" en Contactos y Oportunidades

Botones de acción rápida en los headers de las páginas de Contactos y Pipeline, con Sheet inline y refresco automático de la lista al guardar.

ARCHIVO	OPERACIÓN	DESCRIPCIÓN
`components/quick-action-sheets.tsx`	Modificado	Exports `NewContactButton` + `NewDealButton` con router.refresh()
`app/dashboard/contacts/page.tsx`	Modificado	Botón "Nuevo cliente" en header + fix color foreground para dark mode
`app/dashboard/pipeline/page.tsx`	Modificado	Botón "Nueva oportunidad" en header + fix color foreground para dark mode

Comportamiento: Al guardar, `router.refresh()` recarga los datos del server component sin full-page reload. Los contactos del picker de deals se cargan solo cuando se abre el Sheet (lazy load).

6. Dark Mode

Toggle de tema premium oscuro (slate-950) con persistencia en localStorage via next-themes. Accesible desde /dashboard/settings.

ARCHIVO	OPERACIÓN	DESCRIPCIÓN
`components/theme-provider.tsx`	Modificado	next-themes ThemeProvider con attribute="class", storageKey
`components/theme-toggle.tsx`	Creado	Botón toggle con pill animada, Sun/Moon icons
`app/globals.css`	Modificado	Bloque .dark{} con tokens slate-950/slate-900 + overrides de componentes
`app/layout.tsx`	Modificado	Elimina clase "light" hardcoded; next-themes la gestiona
`app/dashboard/settings/page.tsx`	Modificado	Añade card "Apariencia" con ThemeToggle

Paleta dark: background #020617 (slate-950) · card #0f172a (slate-900) · sidebar #0a0f1e · primary rose-500 `#f43f5e`

Persistencia: localStorage key `advance-crm-theme` · next-themes gestiona SSR con suppressHydrationWarning

31 / 35

32 / 35

Estado actual

Completado · Pendiente · Backlog

✅ Completado

FEATURE	DESCRIPCIÓN	ARCHIVOS CLAVE
Acciones rápidas sidebar	3 Sheets: nuevo cliente, nueva oportunidad, seguimiento	quick-action-sheets.tsx, sidebar.tsx
Alta de propiedades	CRUD completo + upload fotos a Supabase Storage	property-sheet.tsx, properties-client.tsx
WhatsApp directo	Composer + Meta Cloud API + links tel:/wa.me/	api/messages/route.ts, whatsapp-composer.tsx
Propuestas PDF	Selección de propiedades + Puppeteer + branding RE/MAX	api/pdf/route.ts, lib/pdf-template.ts
Ava matching por precio	Tool use Claude + filtro por presupuesto + SQL migration	agent/brain.py, agent/tools.py, 0003_contact_budget_fields.sql
Dark mode	next-themes + tokens slate-950 + toggle en settings	theme-provider.tsx, theme-toggle.tsx, globals.css
Botones "Nuevo" en páginas	NewContactButton en /contacts, NewDealButton en /pipeline	quick-action-sheets.tsx, contacts/page.tsx, pipeline/page.tsx
Vista detalle /contacts/[id]	Perfil completo: deals, tareas, conversación WhatsApp inline	contacts/[id]/page.tsx, contact-actions.tsx
Editar/eliminar contactos	Sheet de edición + delete con confirm por fila en tabla	contact-sheet.tsx, contacts-table.tsx
Editar/eliminar deals	Sheet de edición + delete en hover sobre cards del pipeline	deal-sheet.tsx, pipeline-client.tsx
Drag & drop pipeline	@dnd-kit: mover deals entre columnas, update optimista en Supabase	pipeline-client.tsx, package.json
Seguridad API	Middleware, auth defense-in-depth, UUID/phone validation, prompt injection fix	middleware.ts, api/messages, api/ava, api/pdf
Realtime conversaciones	ConversationsClient con postgres_changes subscription, refetch automático	conversations-client.tsx, conversations/page.tsx
Realtime contacto WhatsApp	ContactWhatsApp con INSERT subscription filtrada, auto-scroll al fondo	contacts/[id]/contact-whatsapp.tsx

33 / 35

FEATURE	DESCRIPCIÓN	ARCHIVOS CLAVE
Tasks CRUD	Toggle completado + editar Sheet + eliminar con confirm	tasks-table.tsx, task-sheet.tsx

⏳ Pendiente de ejecutar manualmente

ACCIÓN	DÓNDE	DETALLE
Migración SQL budget fields	Supabase SQL Editor	`migrations/0003_contact_budget_fields.sql` — columnas ya verificadas como existentes vía REST
Crear bucket Supabase Storage	Supabase Dashboard → Storage	Nombre: `property-images` · Acceso: public · Políticas: INSERT para authenticated
Configurar WHATSAPP_TOKEN en .env	`advance-crm/.env.local`	Para envío outbound desde CRM (Feature 3)
Configurar AVA_WEBHOOK_SECRET	`advance-crm/.env.local` + Railway	Mismo valor en ambos entornos. Python agent debe enviar `Authorization: Bearer <secret>`

📋 Backlog (definido, no implementado)

ITEM	PRIORIDAD	DESCRIPCIÓN
Ava — Multimedia (voz, imagen, links)	Alta	Recibir y procesar mensajes multimedia entrantes desde WhatsApp: **Notas de voz:** Descargar audio/ogg desde Meta API → transcribir con OpenAI Whisper → pasar texto a thread de Ava **Imágenes:** Descargar imagen → analizar con GPT-4o Vision → incluir descripción en contexto de Ava **Links:** Detectar URLs en mensaje → extraer título/descripción con og:meta o puppeteer → enriquecer contexto **DB:** Campos `media_url` + `media_type` en tabla `messages` · Migración `0004_messages_media_fields.sql` **CRM:** Mostrar burbujas de audio (player HTML5), thumbnails de imagen y previews de link en el chat del contacto
App móvil Expo	Alta	Agentes en campo — acceso a contactos, pipeline y composer desde iOS/Android
Deploy a Vercel	Media	CI/CD automático desde GitHub. Prerrequisito: variables de entorno, dominio custom, Puppeteer layer.
Calendario de citas	Media	Vista calendar integrada con tareas y seguimientos
Reportes y analítica	Baja	Conversión por etapa, leads por fuente, revenue pipeline
Meta Ads API	Baja	Conectar Marketing API de Meta a /dashboard/ads

34 / 35

35 / 35